

SIRH Prototype

Guide d'installation et de prise en main de l'équipe

Du design system React au design system Angular, puis au prototype : comment tout a été mis en place, et comment chaque membre de l'équipe configure son poste (Git, VS Code, Claude Code) pour réaliser des fonctionnalités sur le dépôt **sirh_prototype**.

Sommaire

1. Vue d'ensemble : trois dépôts, un seul objectif	3
Comment les pièces s'emboîtent	3
2. Comment on est passé de React à Angular	4
2.1 Le choix de la stratégie	4
2.2 Les trois lots, par niveau de dépendance	4
2.3 Les conventions qui en découlent	4
2.4 Les pièges rencontrés (déjà résolus, à connaître)	4
3. Installer les outils sur votre poste	6
3.1 Node.js et npm	6
3.2 Git	6
3.3 Visual Studio Code	6
3.4 Claude Code	6
4. Accès à GitHub entreprise	7
4.1 Prérequis : être membre de l'organisation	7
4.2 Clé SSH : s'identifier sans mot de passe	7
5. Cloner sirh_prototype et installer le design system	8
5.1 Cloner	8
5.2 Authentification au registre privé (GitHub Packages)	8
5.3 Installer	8
5.4 Erreurs d'installation fréquentes	8
6. Lancer le prototype	9
Structure du projet	9
7. Travailler avec Claude Code sur le projet	10
7.1 Ce qui est déjà configuré (rien à faire)	10
7.2 Première session	10
7.3 Demander une page	10
7.4 Écrire une bonne spec	10
7.5 Quand le design system ne suffit pas : GAP-DS	11
8. Travailler à plusieurs sur le même dépôt	12
8.1 Le cycle d'une fonctionnalité	12
8.2 Éviter les conflits	12
9. Pour les mainteneurs du design system	13
9.1 Faire évoluer le design system	13
9.2 Tester un changement du design system avant de le publier	13

9.3 Mise en route initiale du registre (une seule fois)	13
10. Dépannage	14
Lexique	14
Liens	14

1. Vue d'ensemble : trois dépôts, un seul objectif

Le projet SIRH Socium repose sur **trois dépôts GitHub** de l'organisation **Socium-Design**. Comprendre leur rôle évite 90 % des confusions.

Dépôt	Rôle	Technologie	À modifier ?
design_system	Design system d'origine, en React. Reste la référence figée (tokens Figma, composants, comportements).	React, Vite, Storybook	Non (lecture seule)
design_system_angular	Migration complète du design system en Angular 19 : 27 primitifs, 14 composés, 6 templates de page. Publié sous @socium-design/angular-components .	Angular 19, Storybook 10, Tailwind v4, ng-packagr	Par les mainteneurs du design system uniquement
sirh_prototype	L'application : le prototype vivant de tous les produits (Workspace, Perf, Job...). Il consomme le design system Angular comme une dépendance.	Angular 19	Oui — c'est là que toute l'équipe travaille

Comment les pièces s'emboîtent

design_system (React)		design_system_angular		sirh_prototype
-----		-----		-----
référence Figma/tokens	-->	composants Angular reconstruits	-->	pages du SIRH
(figé, lecture seule)		+ Storybook + tests		(npm install du paquet)
		publié : @socium-design/angular-components		

Règle d'or : **une page du prototype n'invente jamais un composant**. Elle assemble des composants du design system. Si un besoin n'est pas couvert, on le signale comme **GAP-DS** (écart du design system) au lieu de bricoler.

2. Comment on est passé de React à Angular

Cette section raconte la méthode, pour que l'équipe comprenne *pourquoi* les composants sont écrits ainsi. Elle n'a pas besoin d'être refaite : le travail est terminé.

2.1 Le choix de la stratégie

- Le produit réel (le SIRH) est en Angular, alors que le design system d'origine était en React : il fallait un équivalent Angular fidèle.
- Décision : un **nouveau dépôt séparé** (`design_system_angular`). Le dépôt React n'est jamais modifié : il sert de référence.
- Chaque composant Angular est **reconstruit à partir du vrai code React** (props, comportements réels), jamais deviné par analogie avec un élément HTML.

2.2 Les trois lots, par niveau de dépendance

Lot	Contenu	Exemples
Lot 1 — primitifs (27)	Composants sans dépendance vers un autre composant du kit.	Button, Avatar, Badge, Tag, Input*, Checkbox, Switch, Tabs, Tooltip, Popover, SideNavigation, AppSwitch
Lot 2 — composés (14)	Composants qui assemblent des primitifs.	Menu, Select, MultiSelect, Dialog, Drawer, CardGrid, DataTable, HeaderApp
Lot 3 — templates (6)	Pages complètes.	AppShell, PageHome, PageList, PageDetails, PageForm, PageProfile

2.3 Les conventions qui en découlent

- **Composants standalone** (aucun NgModule), signaux (`input()`, `output()`, `model()`), nouvelle syntaxe `@if / @for`.
- **Sélecteur d'attribut** quand le composant EST un élément natif (`<button socButton>`) : tous les attributs natifs fonctionnent. Sinon composant élément (`<soc-card>`).
- **Slots** (équivalent des props React « `ReactNode` ») : on range le contenu avec un attribut marqueur, par ex. `<svg socButtonLeftIcon>`.
- **Champs de formulaire** : tous compatibles `formControl`, `formControlName` et `ngModel`.
- **Styles** : tokens de design en variables CSS, polices auto-hébergées, une seule feuille `styles.css` compilée et livrée avec le paquet.

2.4 Les pièges rencontrés (déjà résolus, à connaître)

Piège	Conséquence	Solution en place
Tailwind v4 et la compilation Angular/Storybook	Aucune classe utilitaire générée, ou erreur au démarrage.	Configuration webpack dédiée dans Storybook + directives <code>@source</code> .
Mémoire de Node pendant le build de Storybook	Le build plante (OOM).	<code>NODE_OPTIONS=-max-old-space-size=6144</code> intégré aux scripts.
Polices non chargées	Texte en police système.	<code>@font-face</code> embarqués dans la feuille de style.

Piège	Conséquence	Solution en place
Un même slot déclaré deux fois dans un template	Contenu projeté perdu en silence.	Un seul <ng-content> par sélecteur ; règle documentée.
Chaque composant embarquait tout Tailwind	Paquet de 35 Mo.	Une seule feuille compilée : 1,7 Mo.

Qualité

Le design system est vérifié par 113 tests unitaires (Karma/Jasmine), par ses 186 stories Storybook rejouées sans erreur console, et par un contrôle visuel dans le navigateur.

3. Installer les outils sur votre poste

À faire une seule fois par machine. Les commandes sont données pour macOS ; Windows : utilisez Git Bash ou WSL, le reste est identique.

3.1 Node.js et npm

```
node -v      # doit afficher v20 ou plus récent (v22 recommandé)
npm -v
```

Si Node n'est pas installé : installez la version LTS depuis nodejs.org, ou utilisez un gestionnaire de versions (nvm, fnm).

3.2 Git

```
git --version
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@socium.link"
```

Nom et email ne sont pas une authentification

`user.name` et `user.email` ne sont que des **étiquettes** écrites dans chaque commit. Ils ne donnent aucun droit : c'est la clé SSH (ou le jeton) qui prouve votre identité à GitHub. Utilisez l'email de votre compte GitHub pour que vos commits vous soient attribués.

3.3 Visual Studio Code

1. Installez VS Code depuis code.visualstudio.com.
2. Ouvrez le dossier du projet : à l'ouverture, VS Code propose d'installer les extensions recommandées (**Angular Language Service**, **Claude Code**, **EditorConfig**) — acceptez.
3. Optionnel : activez la commande code dans le terminal (palette de commandes : « Shell Command: Install 'code' command in PATH »).

3.4 Claude Code

Claude Code est l'assistant qui lit le projet, écrit le code et le vérifie. Il faut un **compte Claude de l'entreprise** (demandez l'invitation à votre administrateur).

```
# Installation (méthode recommandée par Anthropic)
curl -fsSL https://claude.ai/install.sh | bash
# ou : npm install -g @anthropic-ai/claude-code

claude --version      # vérifie l'installation
claude                # premier lancement : connexion avec le compte de l'entreprise
```

Autres interfaces

Claude Code existe aussi comme extension VS Code et dans l'application de bureau Claude (onglet Code). Toutes lisent les mêmes fichiers de configuration du projet (`CLAUDE.md`, `.claude/`) : le comportement est identique.

4. Accès à GitHub entreprise

4.1 Prérequis : être membre de l'organisation

Demandez à un propriétaire de l'organisation **Socium-Design** de vous inviter avec votre compte GitHub personnel, puis acceptez l'invitation reçue par email. Sans cela, le dépôt et le design system sont invisibles (erreurs « Repository not found »).

4.2 Clé SSH : s'identifier sans mot de passe

1. Générez une clé : `ssh-keygen -t ed25519 -C "prenom.nom@socium.link"` (Entrée pour l'emplacement par défaut, choisissez une phrase secrète).
2. Démarrez l'agent et ajoutez la clé : `eval "$(ssh-agent -s)" && ssh-add ~/.ssh/id_ed25519`.
3. Copiez la clé publique : `pbcopy < ~/.ssh/id_ed25519.pub` (macOS).
4. GitHub : **Settings, SSH and GPG keys, New SSH key**, collez la clé. Si l'organisation impose le SSO : **Configure SSO, Authorize** pour Socium-Design.
5. Testez : `ssh -T git@github.com` doit répondre « Hi <votre-pseudo>! You've successfully authenticated ».

Plusieurs comptes GitHub sur la même machine

Si vous avez un compte personnel ET un compte pro, créez une clé par compte et un alias dans `~/.ssh/config` (par ex. Host `github.com-socium` avec IdentityFile `~/.ssh/id_socium`), puis clonez avec `git@github.com-socium:Socium-Design/...`. Vérifiez toujours `git remote -v` : un mauvais remote pousse vers le mauvais dépôt.

5. Cloner sirh_prototype et installer le design system

5.1 Cloner

```
git clone git@github.com:Socium-Design/sirh_prototype.git
cd sirh_prototype
```

5.2 Authentification au registre privé (GitHub Packages)

Le design system est un paquet **privé** : @socium-design/angular-components, hébergé sur GitHub Packages. Le dépôt contient déjà un fichier .npmrc qui dit à npm où le trouver (*sans aucun secret*). Il vous reste à fournir **votre** jeton personnel :

1. GitHub : **Settings, Developer settings, Personal access tokens, Tokens (classic), Generate new token (classic)**.
2. Cochez la permission **read:packages** (rien de plus). Donnez-lui une durée de vie raisonnable. Si SSO : **Configure SSO, Authorize** pour Socium-Design.
3. Copiez le jeton (il ne sera plus affiché) et ajoutez-le dans votre fichier **personnel** ~/ .npmrc (dossier utilisateur, **hors** du dépôt) :

```
//npm.pkg.github.com/:_authToken=ghp_XXXXXXXXXXXXXXXXXXXXXXXXXX
```

```
chmod 600 ~/.npmrc          # seul vous pouvez le lire
```

Sécurité

Ne collez **jamais** un jeton dans le dépôt, dans un message de commit, une issue, ni dans une conversation avec Claude. S'il fuit : révoquez-le immédiatement dans GitHub et générez-en un nouveau.

5.3 Installer

```
npm install
```

npm télécharge Angular, le design system (@socium-design/angular-components) et ses dépendances. Vérification : ls node_modules/@socium-design/angular-components doit lister styles.css, docs/, etc.

5.4 Erreurs d'installation fréquentes

Message	Cause	Correction
E401 Unauthorized / 401	Jeton absent, expiré ou mal recopié.	Revoir ~/.npmrc (une seule ligne, sans espace) ; régénérer le jeton.
E403 Forbidden	Jeton sans read:packages, SSO non autorisé, ou pas membre de l'organisation.	Ajouter la permission ; autoriser le SSO ; demander l'accès au dépôt.
E404 Not Found @socium-design/...	Mauvais registre, ou paquet pas encore publié.	Vérifier le fichier .npmrc du dépôt ; demander à l'administrateur si la version a été publiée.
Permission denied (publickey)	Clé SSH non ajoutée à GitHub / à l'agent.	Reprendre l'étape 4.2 ; tester ssh -T git@github.com.

6. Lancer le prototype

```
npm start          # http://localhost:4200 (rechargement automatique)
npm run build      # build de production (dist/sirh-prototype/)
npm run test:ci    # tests en headless (npm test = mode watch dans Chrome)
```

Ouvrez **http://localhost:4200** : vous voyez l'application complète — barre d'applications à gauche (AppSwitch), en-tête (entreprise, aide, langue, avatar) et menu latéral. Cliquez « Employés » dans le menu Workspace : c'est la **page de référence** (liste recherchable et paginée) qui sert de modèle.

Pages « bientôt disponible »

Un item de menu ou un produit sans page affiche une page « Bientôt disponible » au lieu d'une erreur. Une page se rattache au menu dans un seul fichier : `src/app/core/navigation/navigation.map.ts`.

Structure du projet

```
src/app/
  layout/          shell applicatif (AppSwitch + en-tête + menu), branché sur le routeur
  core/navigation/ navigation.map.ts : item de menu <-> route (à compléter à chaque page)
  core/session/    utilisateur, entreprises, langues (mock)
  products/<produit>/modules/<module>/  une page = un module (page, models, services, tests)
  shared/pages/    pages communes (ex. « bientôt disponible »)
src/mocks/data/    données fictives partagées par tous les produits
docs/              guidelines.md, design-system.md, ce guide
CLAUDE.md  .claude/ configuration de Claude Code pour le projet
```

7. Travailler avec Claude Code sur le projet

7.1 Ce qui est déjà configuré (rien à faire)

Fichier	Rôle
CLAUDE.md	Les règles du projet que Claude relit à chaque session : style Angular, architecture Produit > Module > Fonctionnalité, usage obligatoire du design system, workflow « de la spec à la page », conventions de branches et de commits.
docs/design-system.md	Quel template de page pour quel besoin, patrons de pages, pièges connus.
.claude/commands/nouvelle-page.md	La commande /nouvelle-page qui déroule le workflow complet.
.claude/settings.json	Permissions partagées : commandes sûres autorisées sans confirmation ; modification de node_modules interdite (le design system est en lecture seule).
.claude/launch.json	Lancement de l'aperçu du prototype (npm start, port 4200).
node_modules/@socium-design/.../docs/	Référence de tous les composants (générée depuis le code) que Claude consulte avant d'utiliser un composant.

7.2 Première session

1. Dans VS Code, ouvrez le dossier **sirh_prototype** (ou dans un terminal : `cd sirh_prototype`).
2. Lancez **claude** dans le dossier (ou ouvrez le panneau Claude Code de VS Code). Au premier lancement, acceptez de « faire confiance » au dossier.
3. Vérifiez que Claude a chargé le projet : demandez-lui « Résume les règles de CLAUDE.md et où se trouve la référence du design system ».

7.3 Demander une page

Donnez une **spec** et utilisez la commande :

```
/nouvelle-page Module « Contrats » du produit Workspace : liste des contrats (employé, type CDI/CDD, date de début, statut), recherche, pagination, bouton « Ajouter un contrat ». Données fictives réalistes.
```

Claude va : reformuler la spec, créer la branche `feature/workspace-contrats`, lire la référence des composants, écrire modèle + mock + service + page + route, brancher le menu, écrire un test, lancer le build et les tests, puis **ouvrir l'application et contrôler la page** avant de rendre compte (fichiers créés, éventuels GAP-DS, décisions à valider).

7.4 Écrire une bonne spec

Rubrique	À préciser	Exemple
Où	Produit et module	Workspace > Contrats
Type de page	Liste, détail, formulaire, profil, accueil	Liste (puis détail au clic)
Données	Champs, valeurs, volumes	Employé, type CDI/CDD, début, fin, statut ; ~25 lignes
Actions	Boutons, filtres, navigation	Ajouter, rechercher, ouvrir le détail

Rubrique	À préciser	Exemple
États	Vide, chargement, erreur	Message « Aucun contrat » si la recherche ne trouve rien
Maquette	Lien Figma ou capture	Lien vers la frame Figma

Relire avant d'accepter

Claude demande une confirmation avant d'exécuter certaines commandes ou de modifier des fichiers sensibles. Lisez ce qu'il propose. Vérifiez la page dans le navigateur vous-même : un build qui passe ne prouve pas que la page est conforme à la spec.

7.5 Quand le design system ne suffit pas : GAP-DS

Si la spec demande quelque chose que le kit ne fait pas (composant, variante, prop, écart visuel), Claude doit le **signaler comme GAP-DS** et proposer un contournement honnête. Transmettez ces GAP-DS aux mainteneurs du design system (dépôt `design_system_angular`, issue GitHub) : ils sont corrigés à la source, jamais contournés par un faux composant.

8. Travailler à plusieurs sur le même dépôt

8.1 Le cycle d'une fonctionnalité

```
git checkout main && git pull                # partir d'un main à jour
git checkout -b feature/workspace-contrats    # feature/<produit>-<module>
# ... développement (avec Claude) ...
npm run build && npm run test:ci              # tout doit être vert
git add <fichiers>                           # préférer des fichiers nommés à « git add . »
git commit -m "Workspace > Contrats : liste des contrats"
git push -u origin feature/workspace-contrats
# puis ouvrir une Pull Request vers main sur GitHub
```

- **Ne jamais pousser directement sur main** : tout passe par une Pull Request relue par un collègue.
- Messages de commit : <Produit> > <Module> : description courte.
- Une branche = un module. Plus la branche est courte, moins il y a de conflits.

8.2 Éviter les conflits

- Les fichiers **partagés** par tous : `navigation.map.ts`, les fichiers `*.routes.ts`, `src/mocks/data/`. Ils reçoivent de petites lignes ajoutées : récupérez souvent main (`git pull --rebase origin main`) pour résoudre tôt.
- **Mocks** : un jeu de données unique et cohérent. Avant de créer une donnée, regardez si elle existe déjà (par ex. les employés) ; ne dupliquez pas.
- En cas de conflit, ne choisissez pas « la mienne » à l'aveugle : gardez les deux ajouts. Demandez à Claude de résoudre en expliquant son choix, puis relancez build et tests.

9. Pour les mainteneurs du design system

9.1 Faire évoluer le design system

1. Cloner `design_system_angular`, `cd packages/angular`, `npm install`.
2. Modifier un composant ; vérifier dans Storybook (`npm run storybook`, port 6006) et lancer `npm test` (113 tests).
3. Construire : `npm run build` (régénère la documentation des composants et la feuille de style).
4. Passer la version dans `packages/angular/angular-components/package.json`.
5. Créer le tag correspondant : `git tag v0.1.1 && git push origin v0.1.1` — la CI (GitHub Actions) teste, construit et **publie** sur GitHub Packages.
6. Dans `sirh_prototype` : `npm install @socium-design/angular-components@latest`, vérifier, et committer `package.json` + `package-lock.json`.

9.2 Tester un changement du design system avant de le publier

```
cd design_system_angular/packages/angular && npm run build
npm pack ./dist/angular-components # crée socium-design-angular-components-X.Y.Z.tgz
cd ../../../../sirh_prototype
npm install --no-save ../design_system_angular/packages/angular/socium-design-angular-components-X.Y.Z.tgz
npm start
```

Pourquoi --no-save

Cette installation locale n'est pas écrite dans `package.json` : on ne committe jamais un chemin local. Un simple `npm install` rétablit la version publiée.

9.3 Mise en route initiale du registre (une seule fois)

1. Fusionner dans `main` du dépôt **design_system_angular** la préparation de la publication (renommage du paquet en `@socium-design/angular-components`, workflows GitHub Actions).
2. Vérifier dans GitHub (Settings, Actions) que les workflows sont autorisés pour l'organisation.
3. Pousser le premier tag : `git tag v0.1.0 && git push origin v0.1.0`. Suivre l'exécution dans l'onglet Actions.
4. Vérifier que le paquet apparaît dans l'onglet **Packages** de l'organisation, et que l'accès « lecture » est donné aux membres (ou rattaché au dépôt).
5. Dans **sirh_prototype**, fusionner la branche `feature/soc le-design-system` après avoir lancé `npm install` et committé le `package-lock.json` mis à jour.

Tant que le paquet n'est pas publié

`npm install` sur `sirh_prototype` échoue en E404 pour tout le monde. Ne fusionnez la branche du prototype qu'après la publication de la v0.1.0.

10. Dépannage

Symptôme	Cause probable	Solution
Une page s'affiche sans style (texte brut)	styles.css du design system non chargée.	Vérifier le tableau « styles » de angular.json (cibles build ET test) ; relancer npm start.
NG8002 : Can't bind to 'x' since it isn't a known property	Composant non importé, ou nom de prop faux.	Ajouter le composant à imports ; relire sa signature dans components.md.
Un élément projeté (icône, badge, actions) n'apparaît pas	Marqueur de slot manquant, ou deux éléments projetés dans un même bloc @if.	Ajouter l'attribut marqueur ; un seul élément par @if.
Le menu surligne le mauvais item / ne navigue pas	Entrée absente de navigation.map.ts.	Ajouter la ligne NAV_ROUTES de l'item.
Port 4200 déjà utilisé	Un autre ng serve tourne.	Fermer l'autre terminal, ou : npm start -- --port 4300.
Les tests ne démarrent pas (Chrome introuvable)	Chrome non installé.	Installer Google Chrome, ou définir CHROME_BIN.
Le build Storybook du design system plante en mémoire	Heap Node trop petit.	Utiliser les scripts npm du dépôt (ils fixent NODE_OPTIONS).

Lexique

Terme	Signification
Primitif / Composé / Template	Niveaux du design system : brique de base / assemblage de briques / page complète.
Slot	Emplacement de contenu projeté dans un composant, repéré par un attribut marqueur (ex. socCardIcon).
GAP-DS	Écart constaté du design system : besoin non couvert, à remonter aux mainteneurs.
Mock	Donnée fictive partagée (src/mocks/data/), servie par un service qui simule une API.
Shell	Cadre applicatif (AppSwitch + en-tête + menu) qui entoure chaque page.

Liens

- Design system React (référence) : github.com/Socium-Design/design_system
- Design system Angular : github.com/Socium-Design/design_system_angular
- Prototype : github.com/Socium-Design/sirh_prototype